

Modules

CRM/ERP Platform Documentation

Dynamic Modules

Modules are the core data containers of the platform. Instead of hard-coded tables, admins define custom modules (like "Contacts", "Projects", "Invoices") with their own fields.

What is a Module?

A **Module** (`DynamicModule`) represents a data entity — equivalent to a database table. Each module has:

- A **name** and **slug** (unique per org, used in URLs: `/m/contacts`)
- **Fields** — typed columns the admin configures
- **Views** — saved presentations (table, kanban, calendar, etc.)
- **Records** — actual data rows

All record data is stored in a JSON blob (`Record.data`), keyed by field name.

Field Types

Category	Types
Text	<code>TEXT</code> , <code>TEXTAREA</code> , <code>RICH_TEXT</code> , <code>EMAIL</code> , <code>PHONE</code> , <code>URL</code>
Numbers	<code>NUMBER</code> , <code>DECIMAL</code> , <code>CURRENCY</code>
Booleans	<code>BOOLEAN</code> , <code>CHECKBOX</code>
Selection	<code>RADIO</code> , <code>DROPDOWN</code> , <code>MULTI_SELECT</code> , <code>STATUS</code>

Category	Types
Date/Time	DATE , DATETIME
Media	FILE , IMAGE , SIGNATURE
Special	USER_SELECT , TAGS , COLOR_PICKER , AUTO_NUMBER , RATING , PROGRESS
Computed	FORMULA , LOOKUP , MIRROR
Relations	GLOBAL_RELATION , INLINE_SUBFORM

Select types (RADIO , DROPDOWN , MULTI_SELECT , STATUS) use FieldOption rows for their choices. Options have label , value , and color .

Creating a Module

1. Go to **Settings** → **Studio**
2. Click **New Module**
3. Set name, icon, color
4. Add fields in the field editor

Or via API:

```
POST /api/v1/modules
{
  "name": "Projects",
  "slug": "projects",
  "icon": "FolderKanban",
  "color": "#7c3aed"
}
```

Field Rules (Conditional Logic)

Field rules control field visibility and values based on conditions.

```
// Rule example:
{
  name: "Show budget if status is active",
  logic: "AND",
  conditions: [
    { field: "status", operator: "equals", value: "active" }
  ],
  actions: [
    { type: "show", target: "budget" }
  ],
  runOnLoad: true,
  stopOnMatch: false
}
```

Rules run client-side on record load and on field change. Managed via `GET/POST/PATCH/DELETE /modules/:moduleId/field-rules`.

Views

Each module can have multiple **Views** — saved configurations for browsing records.

View Type	Description
TABLE	Spreadsheet-like grid
KANBAN	Cards grouped by a status/select field
CALENDAR	Records on a calendar by date field
GALLERY	Card grid with image
TIMELINE	Gantt-style timeline
FORM	Form-based data entry

Views store `filters`, `sorts`, `columns`, `groupBy`, and display `config` as JSON.

Relationships

Modules can be linked to each other via `Relationship` :

```
ONE_TO_ONE: Project — Invoice
ONE_TO_MANY: Contact — Tasks
MANY_TO_MANY: Product — Orders
```

Cross-module lookup fields (`GLOBAL_RELATION`) display data from related modules inline.

Permissions per Module

Each module has permission rows in the `permissions` table, one per role. Users see only modules and records allowed by their resolved permissions. See [Permissions.md](#).

Record Structure

```
{
  "id": "cuid",
  "moduleId": "cuid",
  "organizationId": "cuid",
  "data": {
    "name": "Acme Corp",
    "status": "active",
    "budget": 50000,
    "tags": ["enterprise", "priority"]
  },
  "isDeleted": false,
  "createdAt": "2026-06-01T00:00:00Z"
}
```

The `data` key names match `Field.name` values. The backend validates and coerces types during create/update.

Auto-number Fields

Fields of type `AUTO_NUMBER` auto-increment per module. Format is configurable (prefix + zero-padded number, e.g., `PROJ-0001`). The counter is stored in `Field.settings.autoNumberCounter`.

Formula Fields

`FORMULA` fields compute values from other fields using an expression engine. Example:

```
{total_price} = {quantity} * {unit_price}
```

Formula expressions are stored in `Field.formulaExpression` and evaluated server-side on record fetch.

Soft Deletion

Records are never hard-deleted by default:

- `PATCH /modules/:moduleId/records/:id` with `{ isDeleted: true }` soft-deletes
- `DELETE /modules/:moduleId/records/:id` sets `isDeleted + deletedAt`
- Deleted records can be restored or permanently deleted

The recycle bin UI shows all soft-deleted records.